



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CS304 SOFTWARE ENGINEERING

Yida Tao

taoyd@sustech.edu.cn



PREVIOUSLY

- Programming assignment vs. Software
- Coding vs. Building a software
- Complexity (time, scale, tradeoffs)
- Quality and efficiency

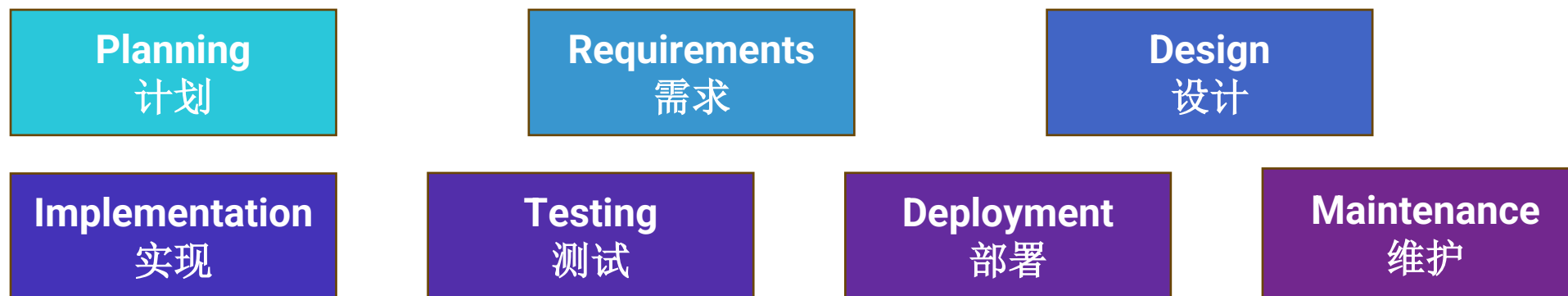


LECTURE 2

- Overview of software process
- Process models
- Agile & Scrum
- DevOps



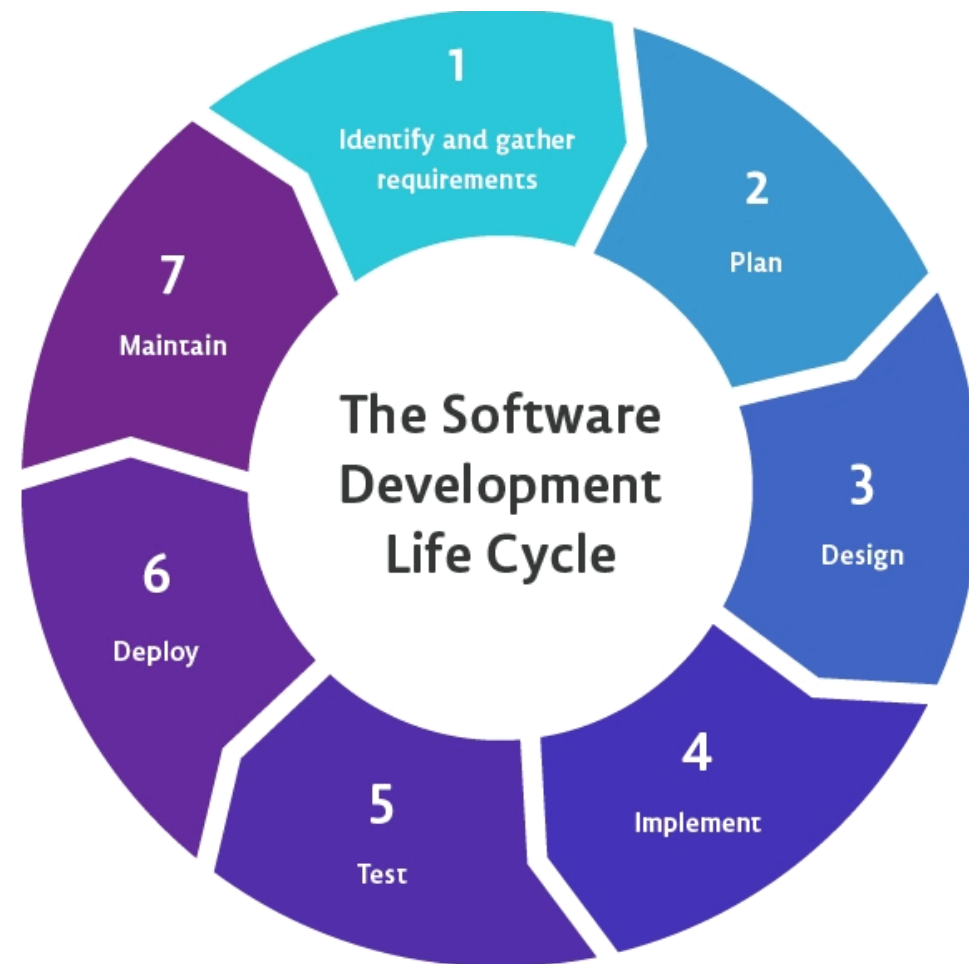
CORE ACTIVITIES FOR BUILDING SOFTWARE



How do we organize these activities to avoid chaos and deliver results?

SOFTWARE PROCESS

Software process (软件过程) is a set of related phases/activities that take place **in certain order**, and leads to the production of a software product.





SOFTWARE PROCESS MODEL

Software process models (or software lifecycle model) were originally proposed to bring order to the chaos of software development, it determines

- The **order** in which core activities are carried out
- How **transitions** between activities are managed

It provides a **template or strategy** for organizing a software project from start to finish.



THE CONTINUUM OF SOFTWARE PROCESS MODELS

Planning is incremental and it is easier to change the process to reflect changing customer requirements.

All of the process activities are planned in advance and progress is measured against this plan.

Agile

Plan-driven

There is no ideal process and most organizations have developed their own software development processes by finding a balance between plan-driven and agile processes.



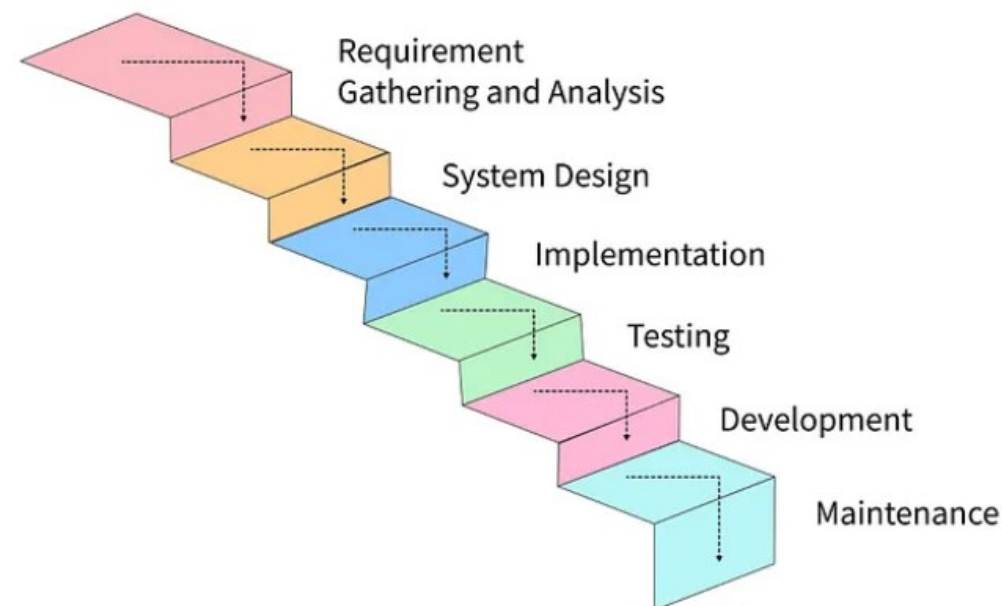
LECTURE 2

- Overview of software process
- Plan-driven process models
- Agile & Scrum
- DevOps



THE WATERFALL MODEL

- A sequential, linear approach.
- Each phase must complete before the next begins (no overlap).
- Flows downwards, like a waterfall.
- Outputs of each phase are approved documents.

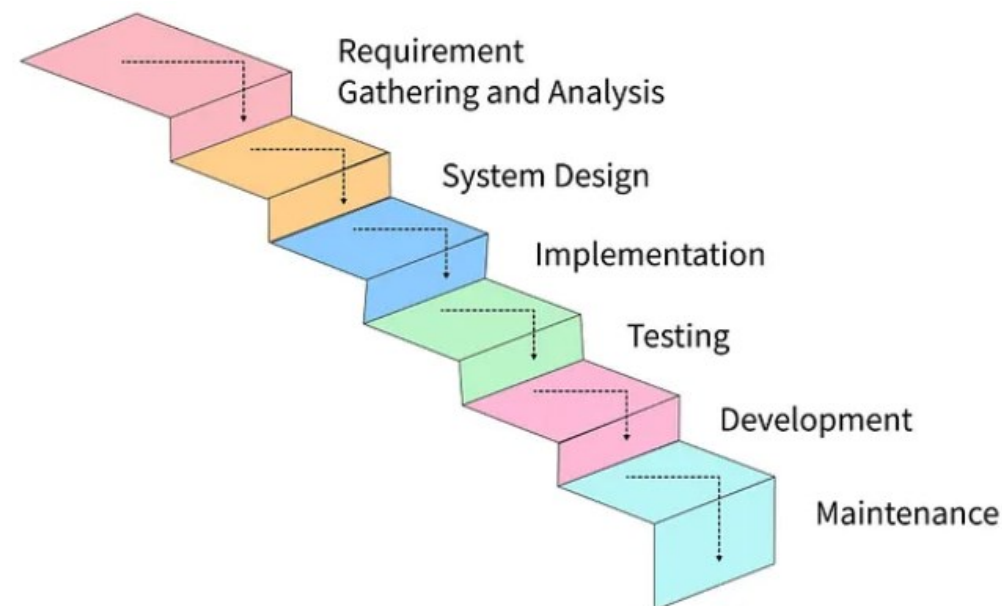


Images sourced online, for teaching purpose only.



THE WATERFALL MODEL

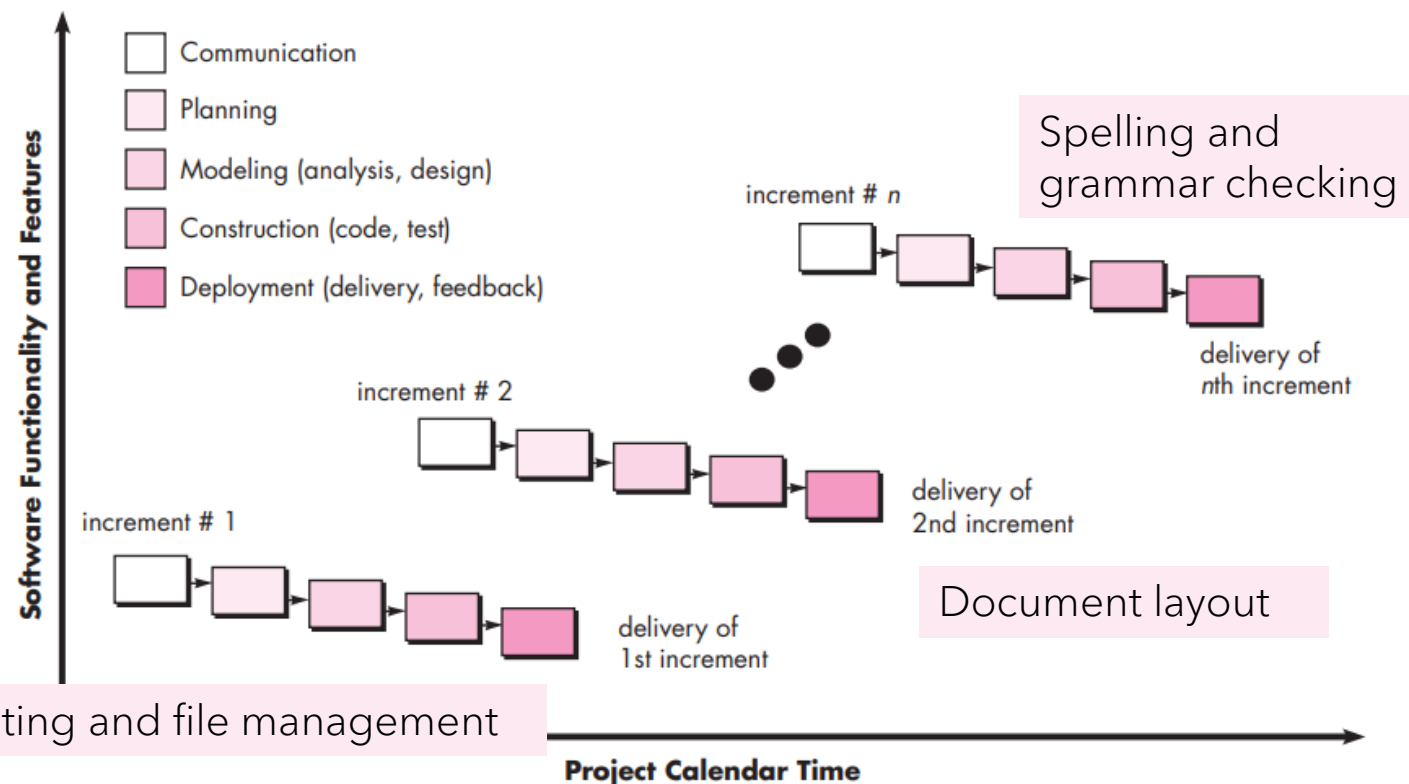
1. Deadlines are **TIGHT**
2. People often **WAIT**
3. Requirements are **UNCLEAR**
4. Requirements keep **CHANGING**



Waterfall is best suited only for **well defined** and **stable** requirements.

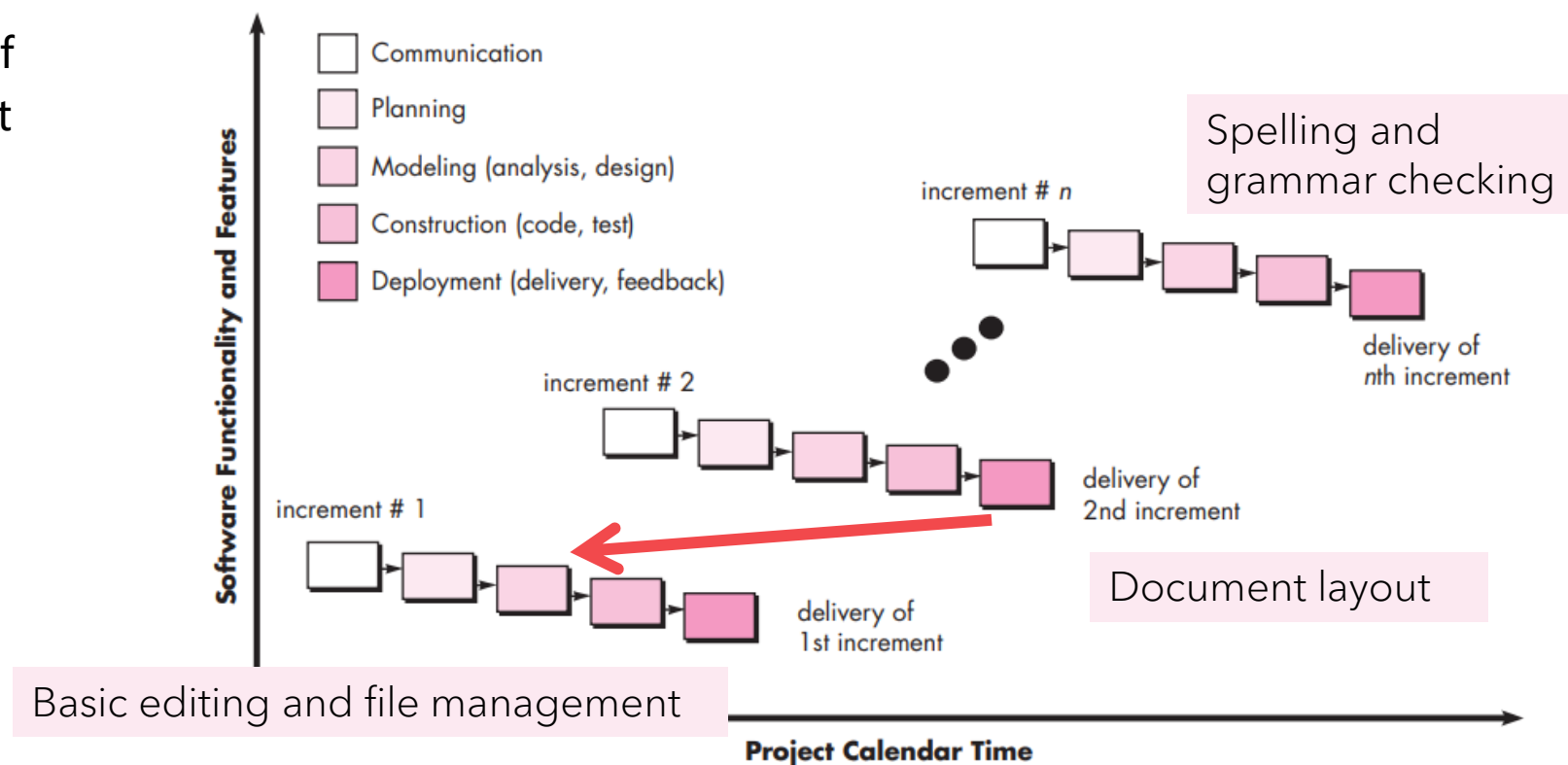
INCREMENTAL PROCESS MODELS

- The incremental model (增量模型) linear and parallel process flows
- Each linear sequence produces deliverable “increments” of the software
- The first increment is often a core product with the basic requirements addressed. Supplementary features are developed in subsequent increments.
- Earlier feedback and easier debugging



INCREMENTAL PROCESS MODELS CONS

- Needs good planning and design of the whole system before breaking it down for incremental builds
- System structure tends to **degrade** as new increments are added.
- **No feedback iterations**





IS SOFTWARE DEVELOPMENT A LINEAR PROCESS?

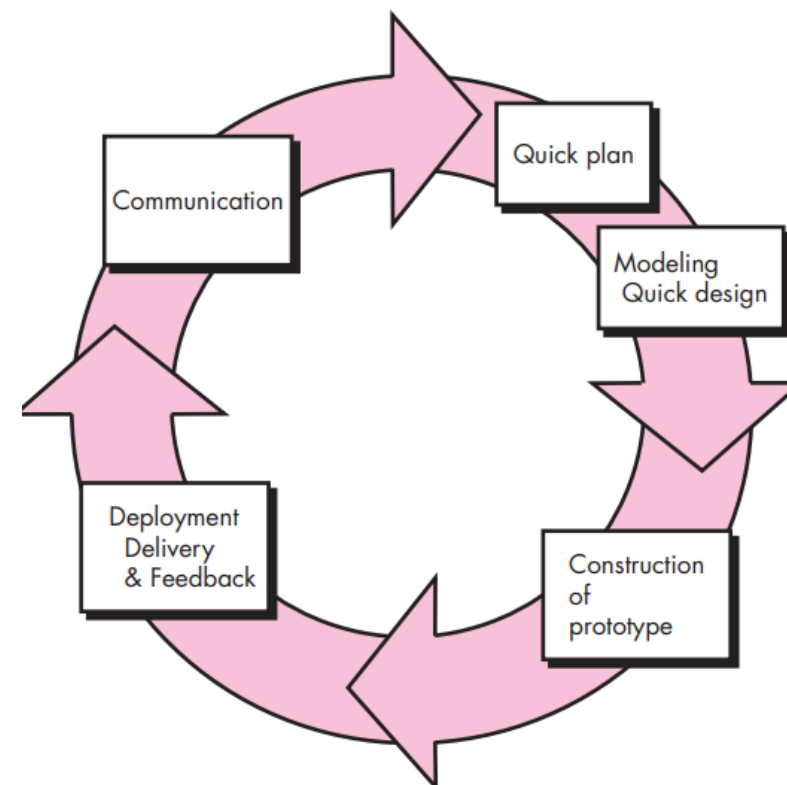


EVOLUTIONARY PROCESS MODELS

- Evolutionary models are **iterative (迭代)**, which is explicitly designed to accommodate a product that evolves over time.
- Early examples: **prototyping (原型)** and **the spiral model (螺旋模型)**

PROTOTYPING

- Used when requirements are general or unclear.
- Quick plan, design, construction, delivery, and feedback loop.
- Customers see a working version quickly.
- Often involves implementation compromises; prototype may be discarded.
- Ideal for UI development or demonstrating feasibility



Software engineering: a Practitioner's Approach by Roger S. Pressman

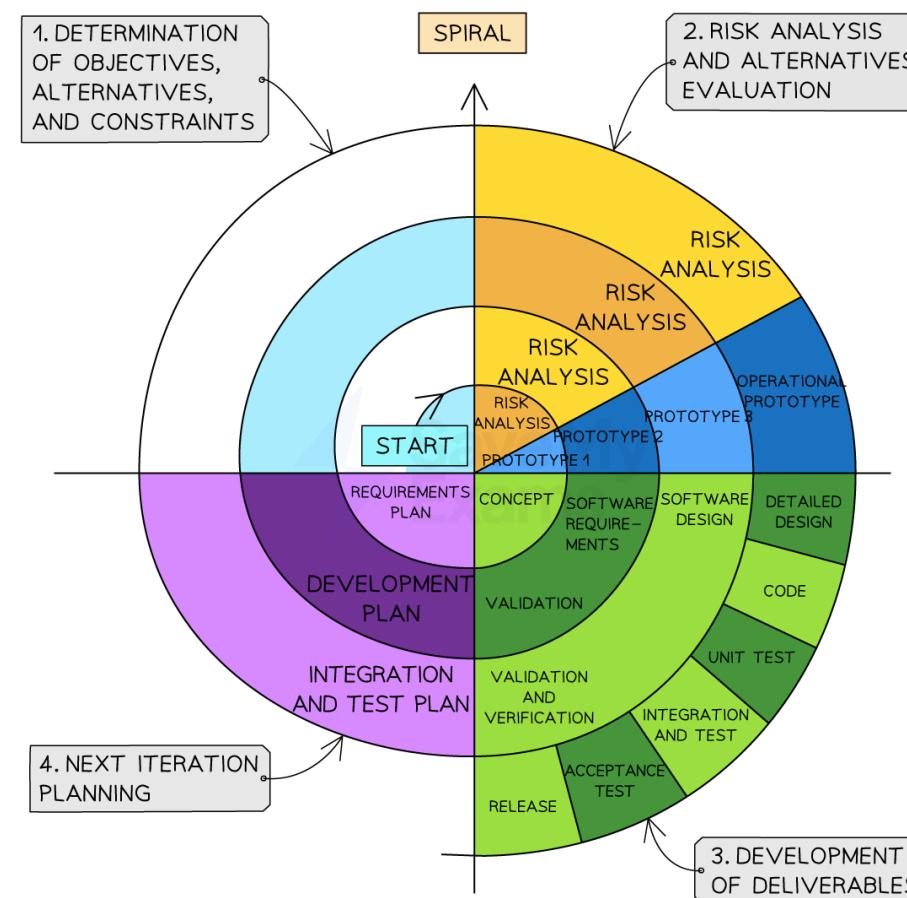
THE SPIRAL MODEL

Key features

- Combines iteration, prototyping, and risk analysis
- Each spiral loop represents one phase of the project

Pros and cons

- Effectively reduces potential risks, good for large, complex and high-risk projects.
- Risk analysis relies heavily on experts, complex and costly.



Images sourced online, for teaching purpose only.



PLAN-DRIVEN SOFTWARE PROCESS MODELS

Software engineers are not robots. They exhibit great variation in working styles; significant differences in skill level, creativity, orderliness, consistency, and spontaneity. Some communicate well in written form, others do not

As consistency in action is a human weakness, high-discipline methodologies are **fragile**

Agile

Plan-driven

FRAGILE

Agile Manifesto

Agile Values

We are uncovering **better ways of developing software by doing it and helping others do it.** Through this work we have come to value:



Individuals and interactions

over

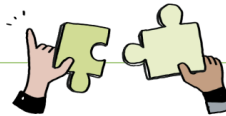
processes and tools



Working software

over

comprehensive documentation



Customer collaboration

over

contract negotiation



Responding to change

over

following a plan

That is, while there is **value in the items on the right**, we **value the items on the left more.**

<https://agilemanifesto.org/>

@SketchingSM
www.sketchingscrummaster.com



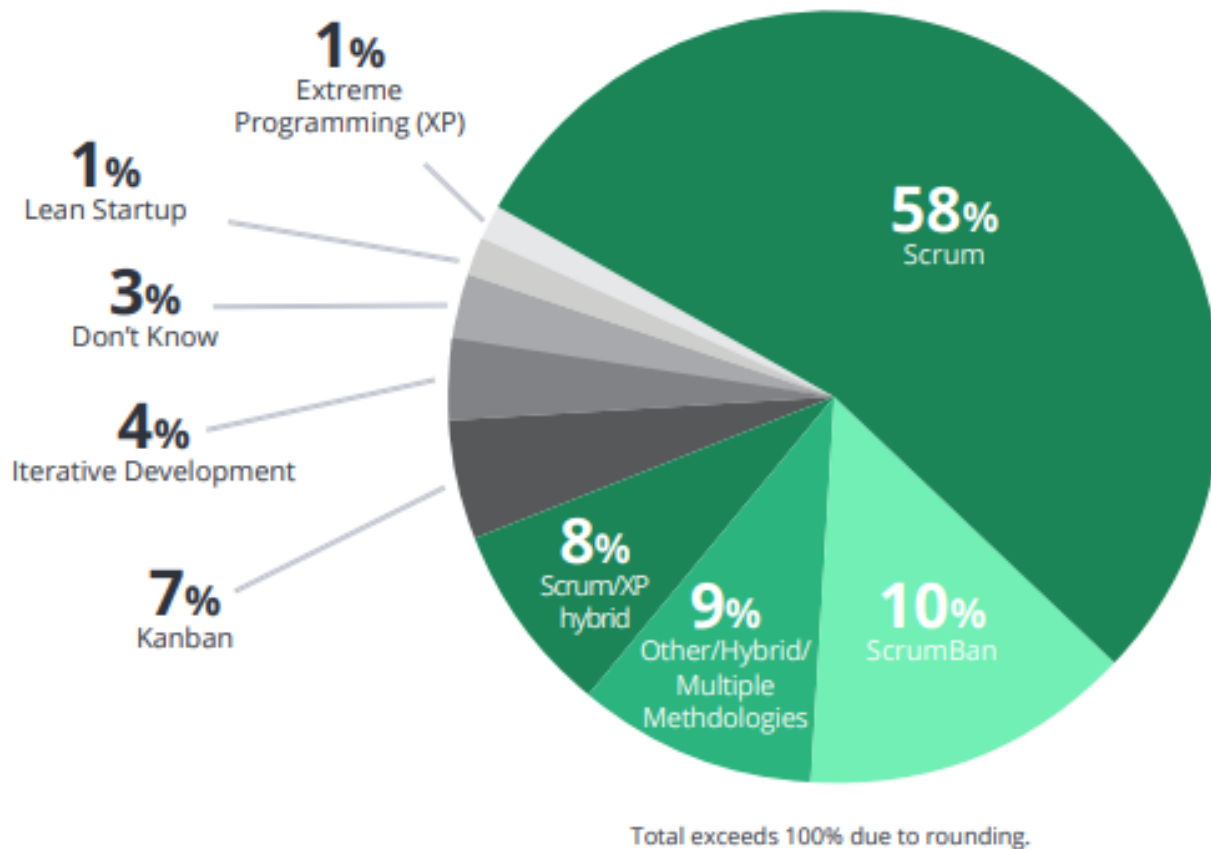
南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

WHAT IS AGILE?

Agile (敏捷), **in general**, is the ability to create and respond to change.

- An agile team delivers work in **small**, but **consumable, increments (增量)**.
- Requirements, plans, and results are **evaluated continuously**
- Teams have a natural mechanism for **responding to change quickly**.

AGILE METHODOLOGY



Source: 14th Annual State of Agile Report

- However, there is NO one right way to implement Agile
- There are many different types of agile methodologies

THE ORIGIN OF SCRUM

- The term “scrum” was borrowed from the game of rugby, to stress the importance of teamwork to quickly adapt to unexpected, complicated situations
- Process of sport games = Process of software development





SCRUM ROLES

Product Owner

Define and adjust product features; Accept or reject work results

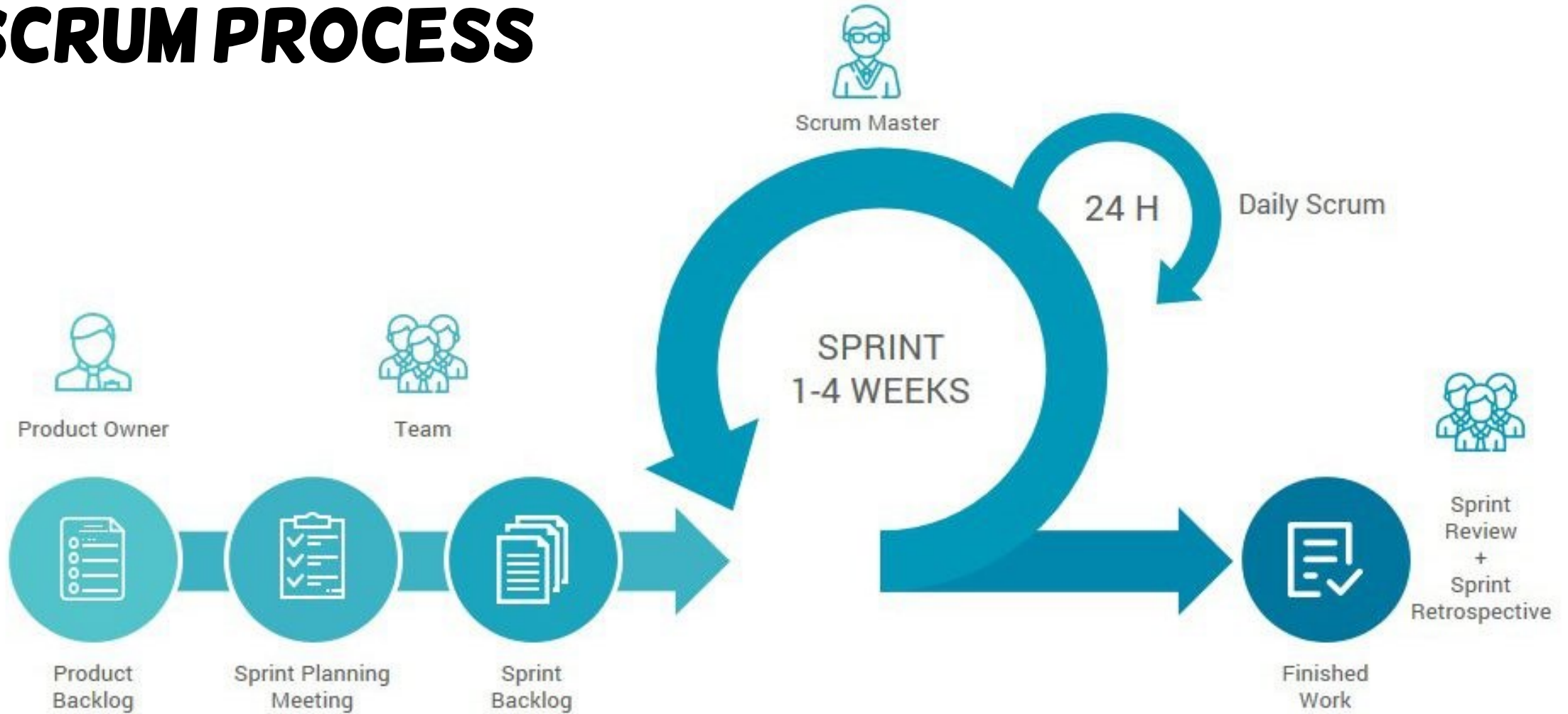
Scrum Master

Coach the team and enact Scrum values and principles

Scrum Team

Cross-functional members like developers, testers, designers (Typically 5-9 size)

SCRUM PROCESS



Sprint (冲刺/迭代周期)

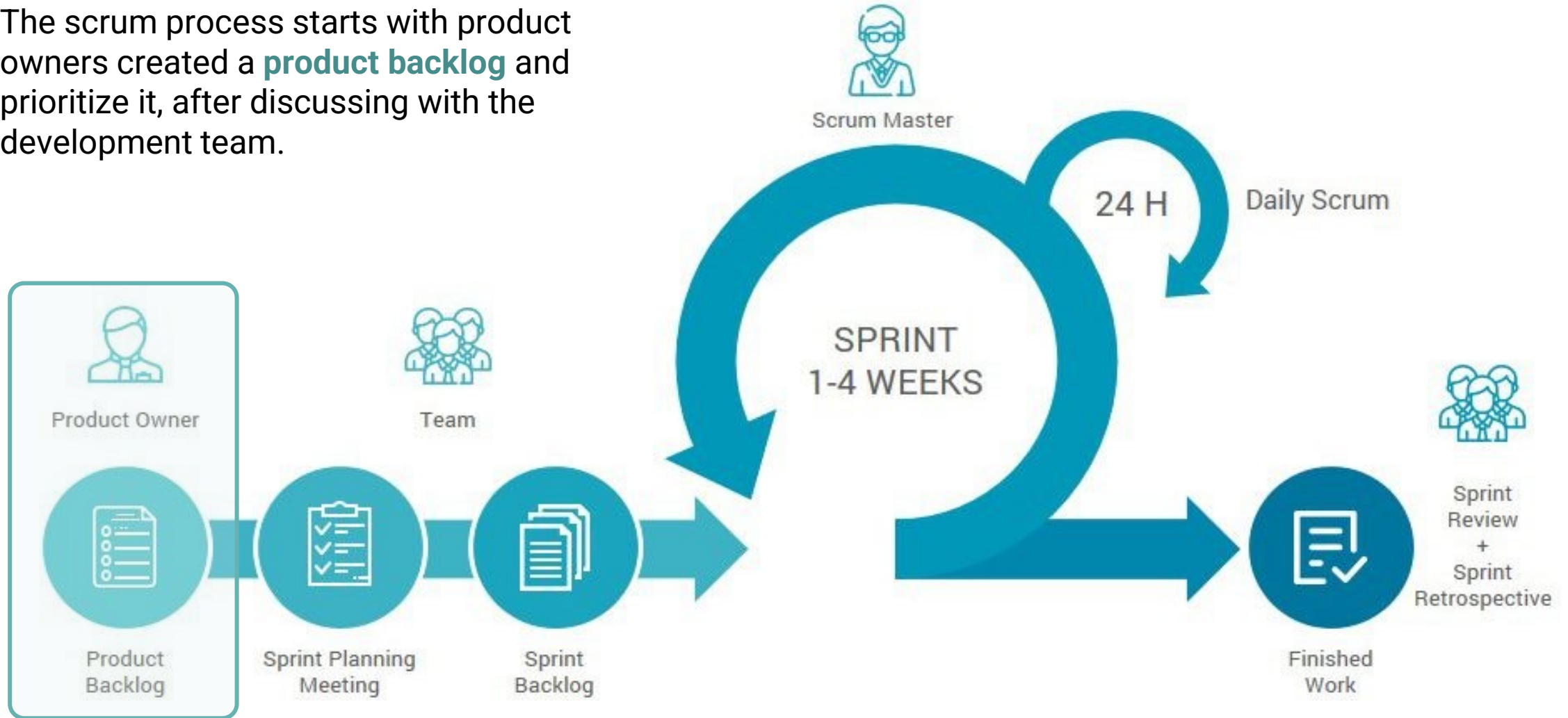
- A short, time-boxed period, typically 1-4 weeks
- The scrum team focuses on delivering a potentially shippable product increment during a sprint

How do people decide what to work on in a SPRINT?



Product Backlog (待办列表)

The scrum process starts with product owners created a **product backlog** and prioritize it, after discussing with the development team.





EXAMPLE OF PRODUCT BACKLOG

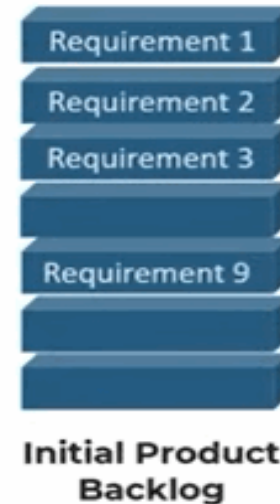
More details on week 3

ToDo List

ID	Story	Estimation	Priority
7	As an unauthorized User I want to create a new account	3	1
1	As an unauthorized User I want to login	1	2
10	As an authorized User I want to logout	1	3
9	Create script to purge database	1	4
2	As an authorized User I want to see the list of items so that I can select one	2	5
4	As an authorized User I want to add a new item so that it appears in the list	5	6
3	As an authorized User I want to delete the selected item	2	7
5	As an authorized User I want to edit the selected item	5	8
6	As an authorized User I want to set a reminder for a selected item so that I am reminded when item is due	8	9
8	As an administrator I want to see the list of accounts on login	2	10
Total		30	

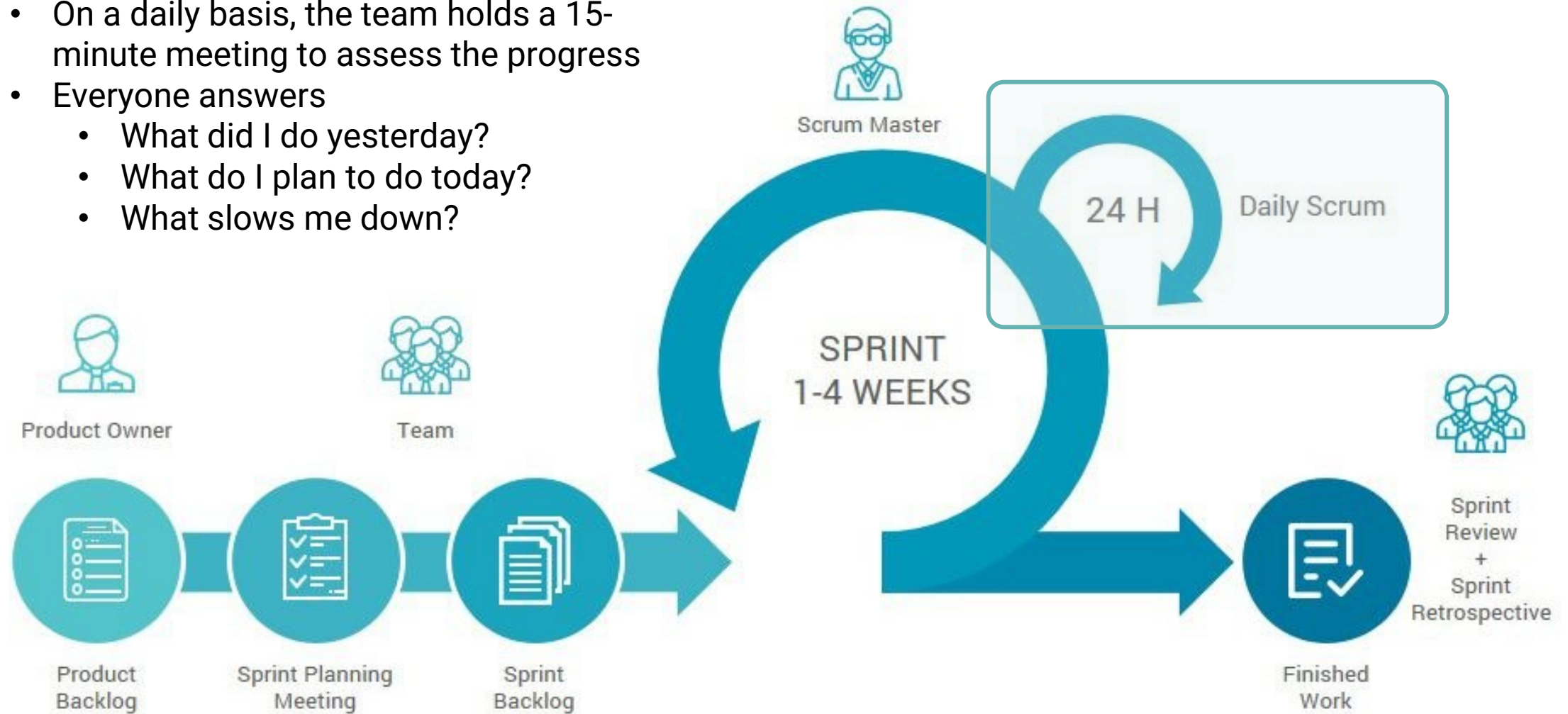
Sprint Backlog

Next, the whole team attend **the Sprint Planning Meeting** to select the items to deliver in the next sprint, which forms the **Sprint Backlog**.



Daily Scrum

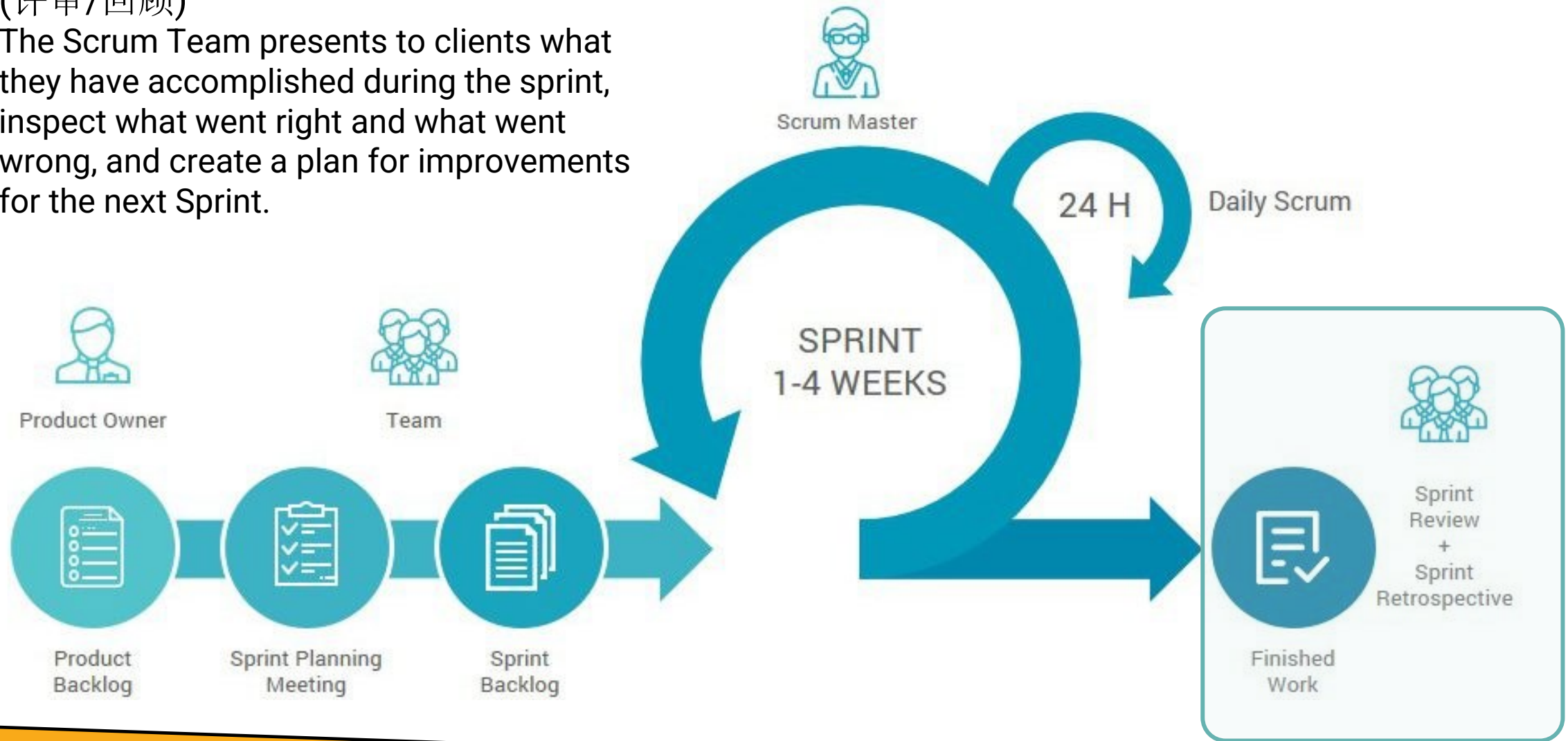
- On a daily basis, the team holds a 15-minute meeting to assess the progress
- Everyone answers
 - What did I do yesterday?
 - What do I plan to do today?
 - What slows me down?



Sprint Review & Retrospective

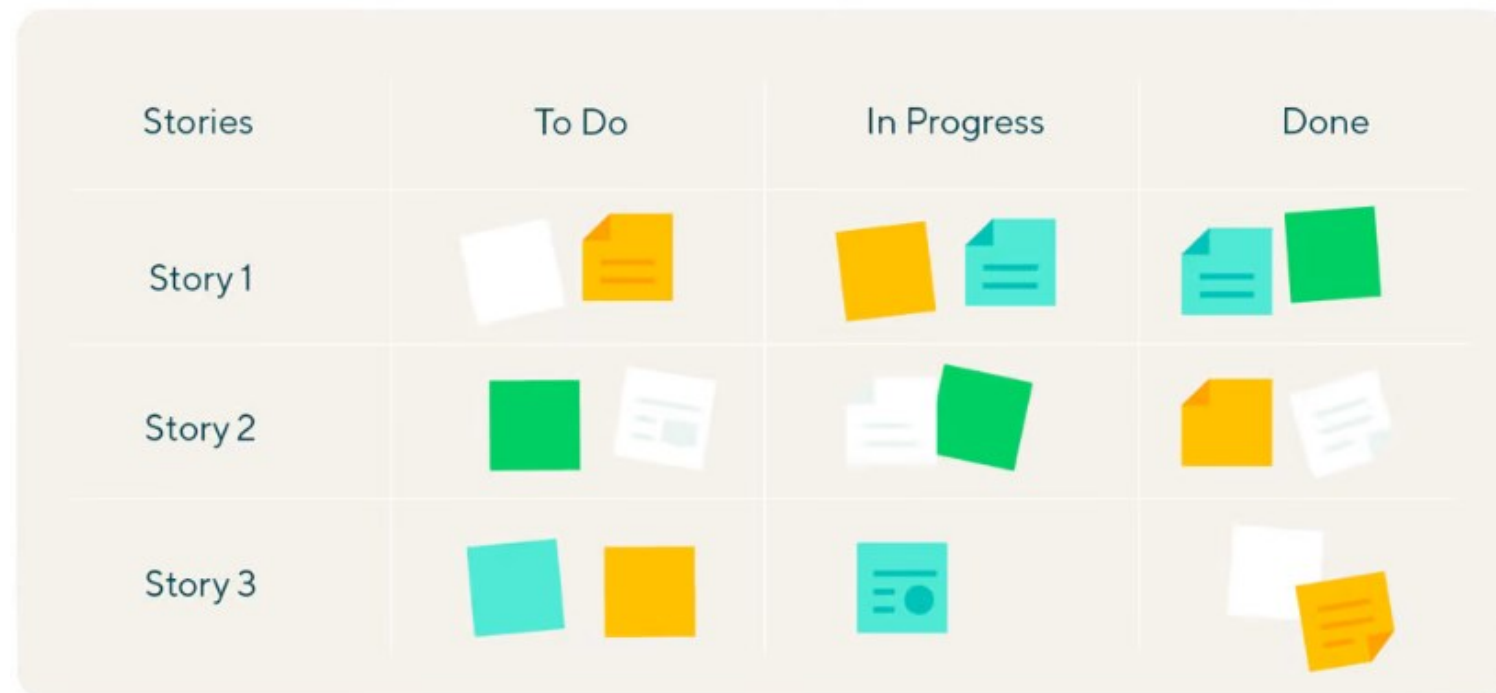
(评审/回顾)

The Scrum Team presents to clients what they have accomplished during the sprint, inspect what went right and what went wrong, and create a plan for improvements for the next Sprint.

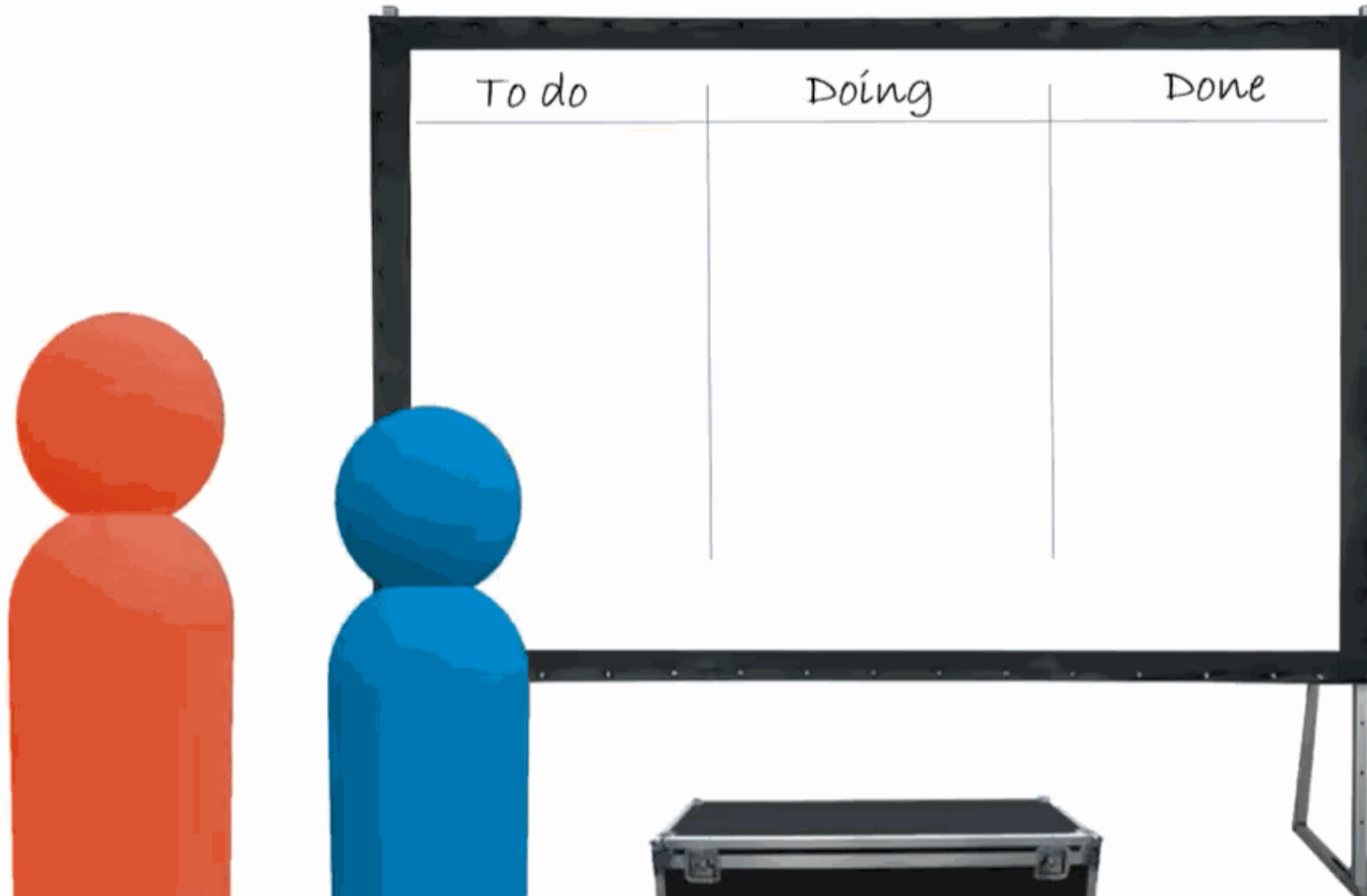


SCRUM BOARDS

- A Scrum Board is a tool that helps Scrum Teams make Sprint Backlog items visible.
- The board is traditionally divided up into three categories: To Do, Work in Progress and Done.

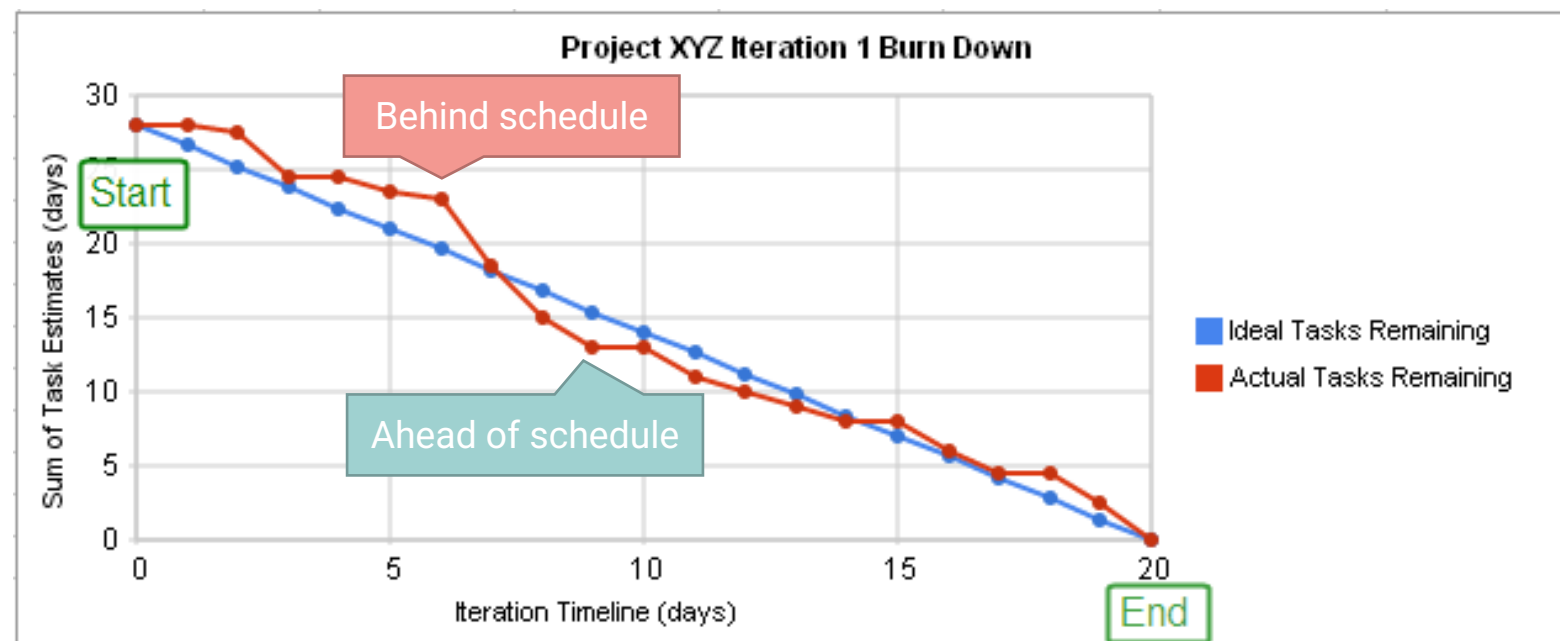


The scrum master



BURNDOWN CHARTS

- A graph showing the amount of work remaining over time.
- The backlog or effort remaining is often on the Y-axis, with time in the sprint along the X-axis
- Useful for predicting whether all work in the backlog will be completed on time





SCRUM BOARD AND BURNDOWN CHARTS IN ACTION

<https://www.scruminc.com/sprint-burndown-chart/>



SCRUM SUMMARY

Roles

Product Owner
Scrum Master
Scrum Team

Ceremonies

Sprint Planning
Daily Scrum
Sprint Review
Sprint Retrospective

Artifacts

Product Backlog
Sprint Backlog
Burndown Charts

SCRUM CASE STUDY



The adoption of Scrum by Google's marketing team led to a **30%** improvement in campaign effectiveness and a **20%** improvement in teamwork.



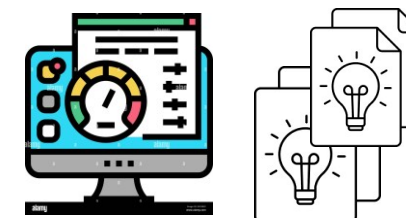
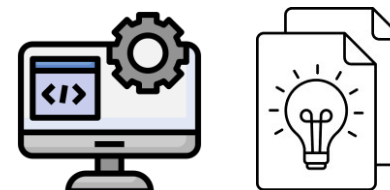
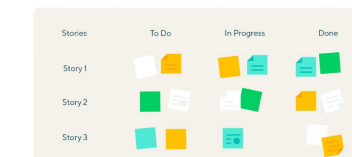
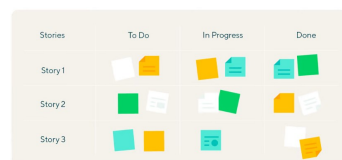
Scrum was adopted by Tesla for their autonomous driving project, which led to a **15%** acceleration of the development cycle and a **10%** decrease in errors.



Cisco switch from waterfall to agile for the billing platform, which led to **40%** decrease in defect rate and **14%** increase in defect removal.



APPLYING SCRUM IN OUR TEAM PROJECT



Week 1
Team up

Week 5
Proposal

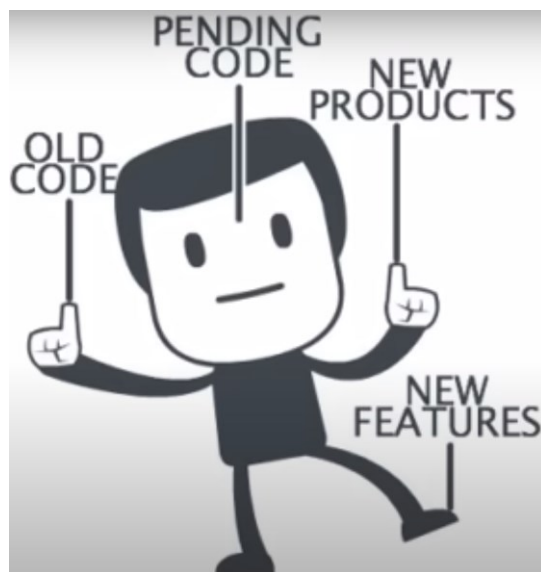
Week 9
Sprint 1

Week 16
Sprint 2



LECTURE 2

- Overview of software process
- Process models
- Agile & Scrum
- DevOps

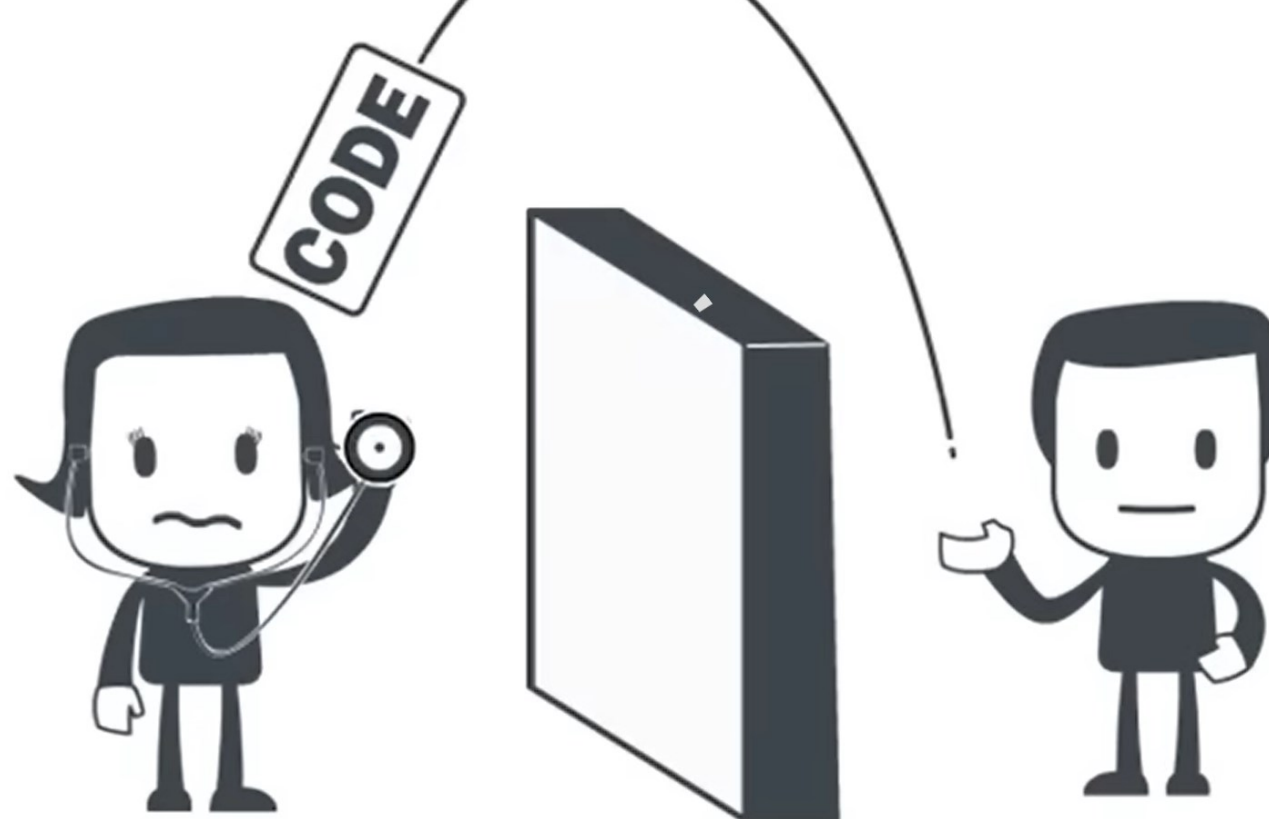


Dev (开发团队) responsibility:

- Design
- Develop
- Test
- Document

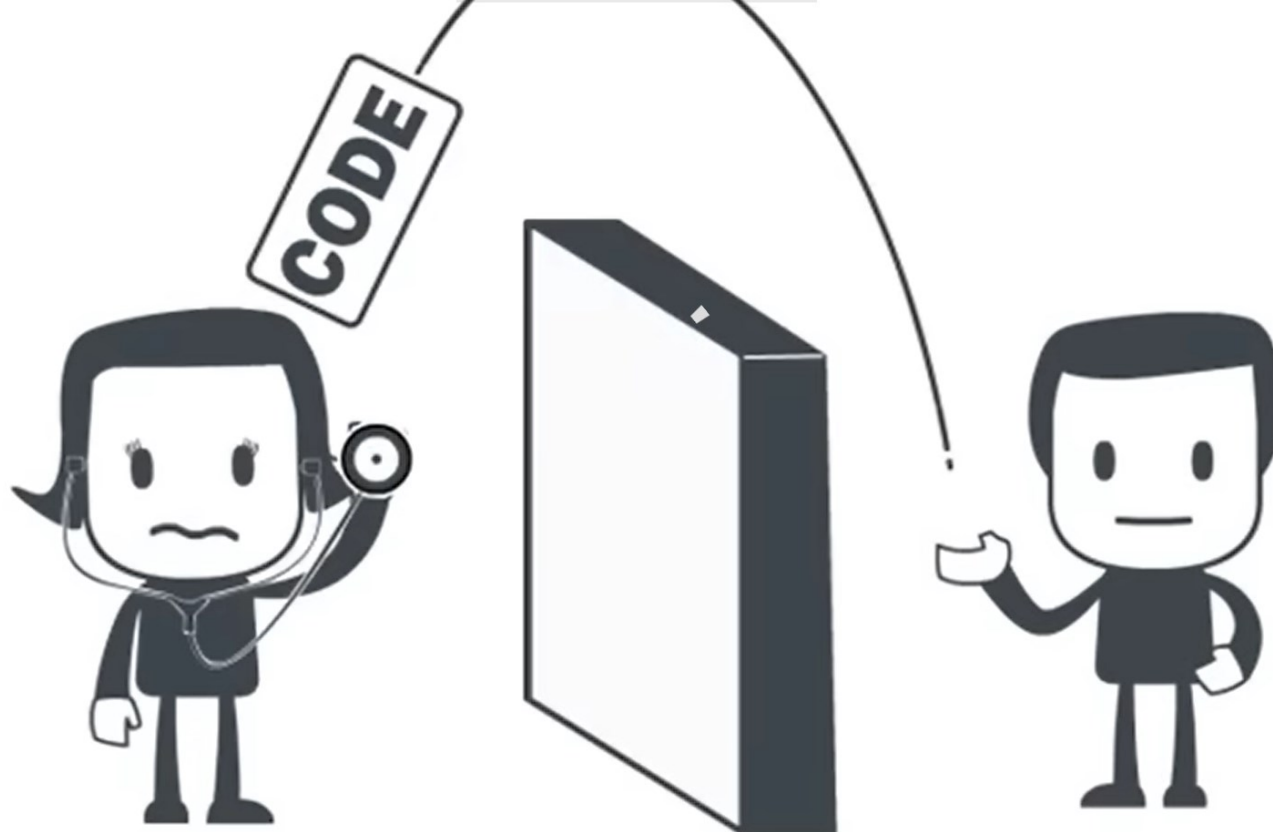
Ops (运维团队) responsibility:

- Deploy
- Release
- Stability of service
- Manage infrastructure
- Maintain & Feedback



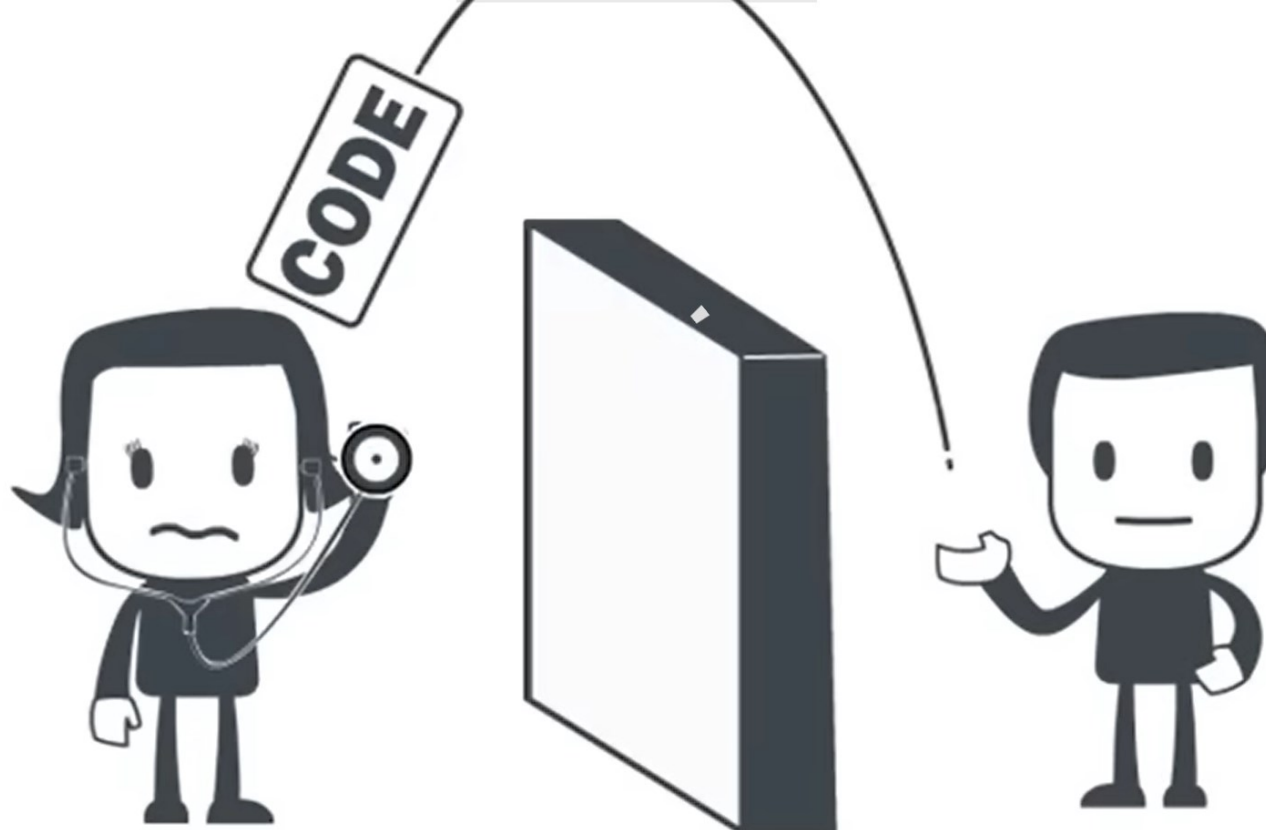
In traditional IT operations, Dev and Ops teams typically work **sequentially** and **individually**

Problems?



Problem 1: “It works on my machine. It’s your problem now.”

- In traditional software organization, dev-to-ops handoffs are typically one-way, often limited to a few scheduled times in an application's release cycle
- Once in production, the operations team take charge



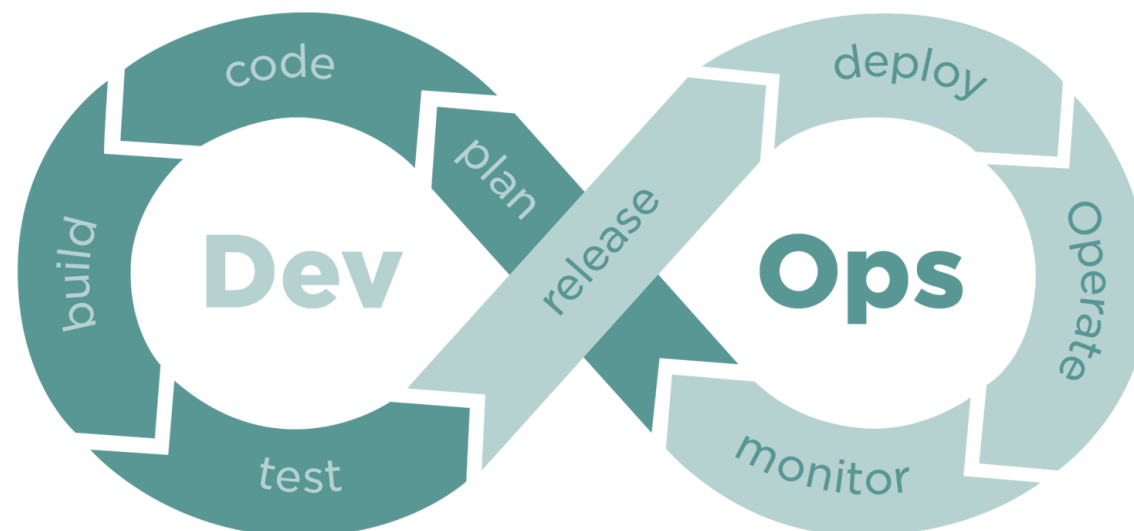
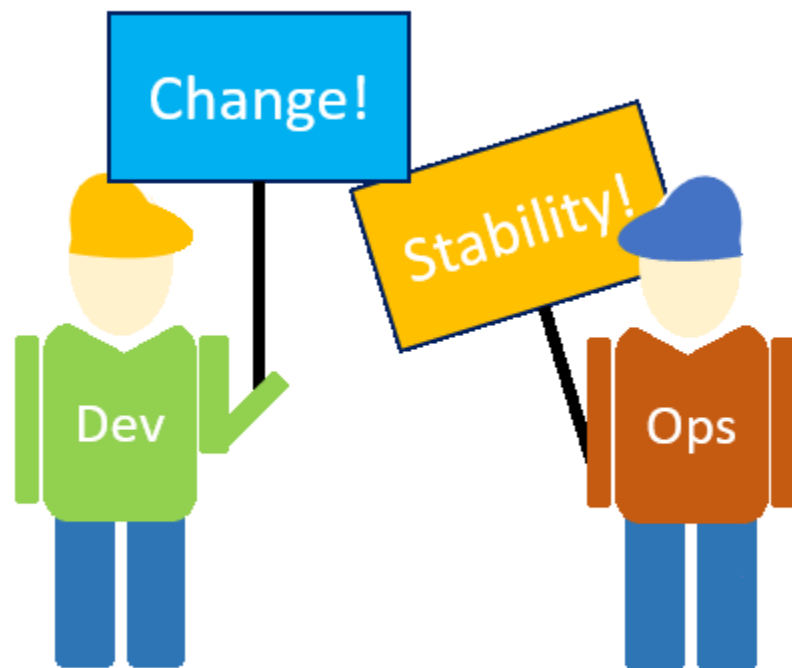
Problem 2: slow feedback, long time to ship

- If there are bugs in the code, the virtual assembly line of Devs-to-QAs-to-OPs is revisited with a patch, with each team waiting on the other for next steps, which might take weeks
- Such a model comes with significant overhead and extends the timeline



Reason: Conflicting objectives for Devs and Ops

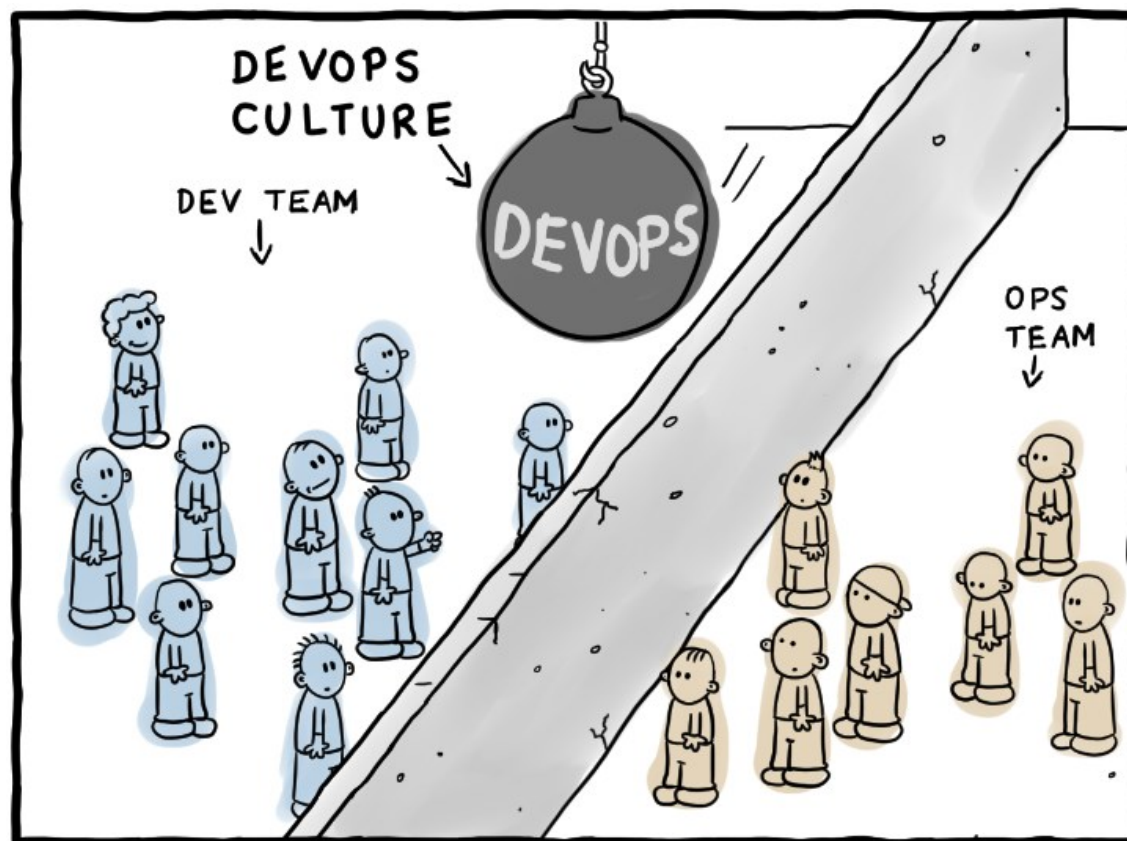
Image source: <https://ralfneubauer.info/devops-gifs-reactions/>



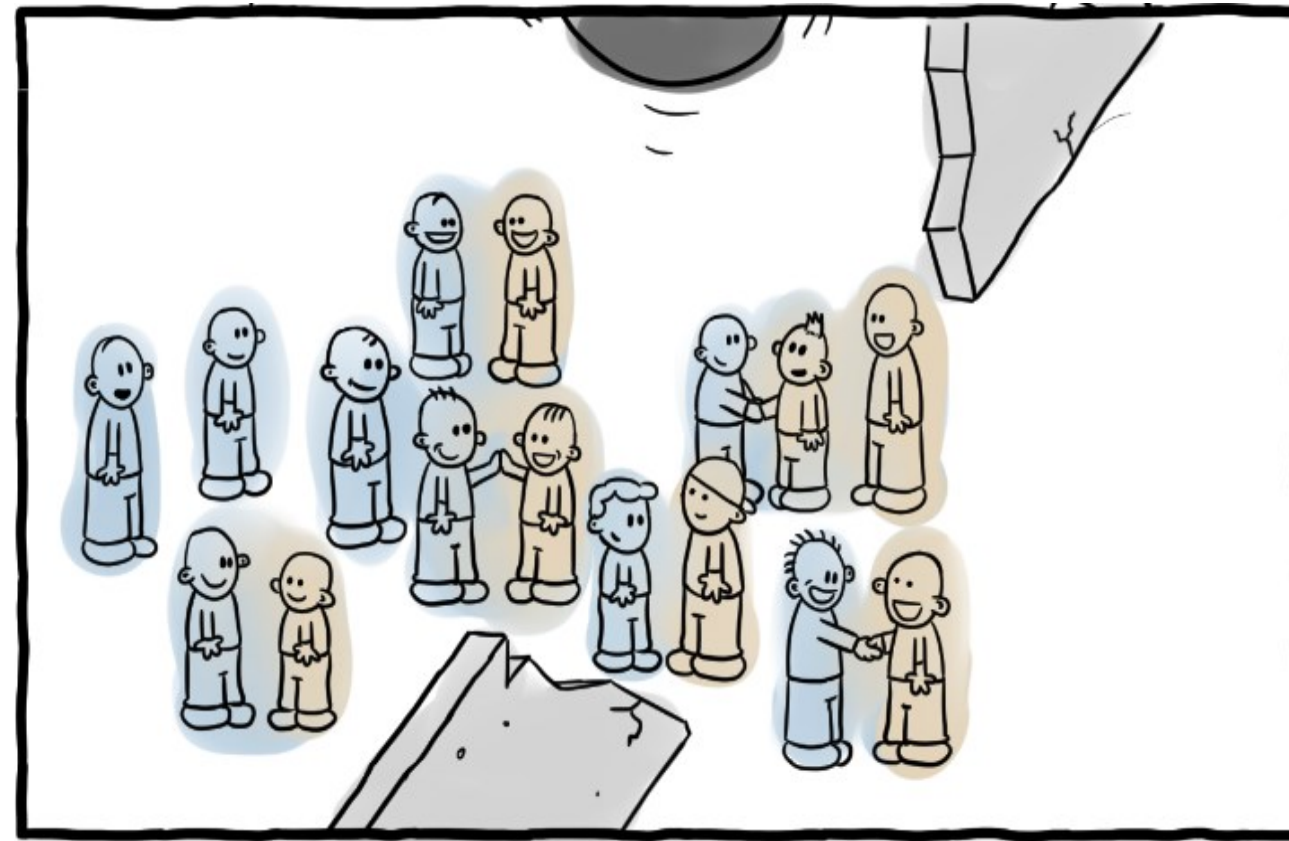
DevOps: Dev and Ops working closely and collaboratively with the same objective

Image source: <https://ralfneubauer.info/devops-gifs-reactions/>

BEFORE/AFTER DEVOPS



<https://turnoff.us/geek/devops-explained/>





DEVOPS

A culture, practice, and set of tools that integrate development (Dev) and operations (Ops).

Key principles

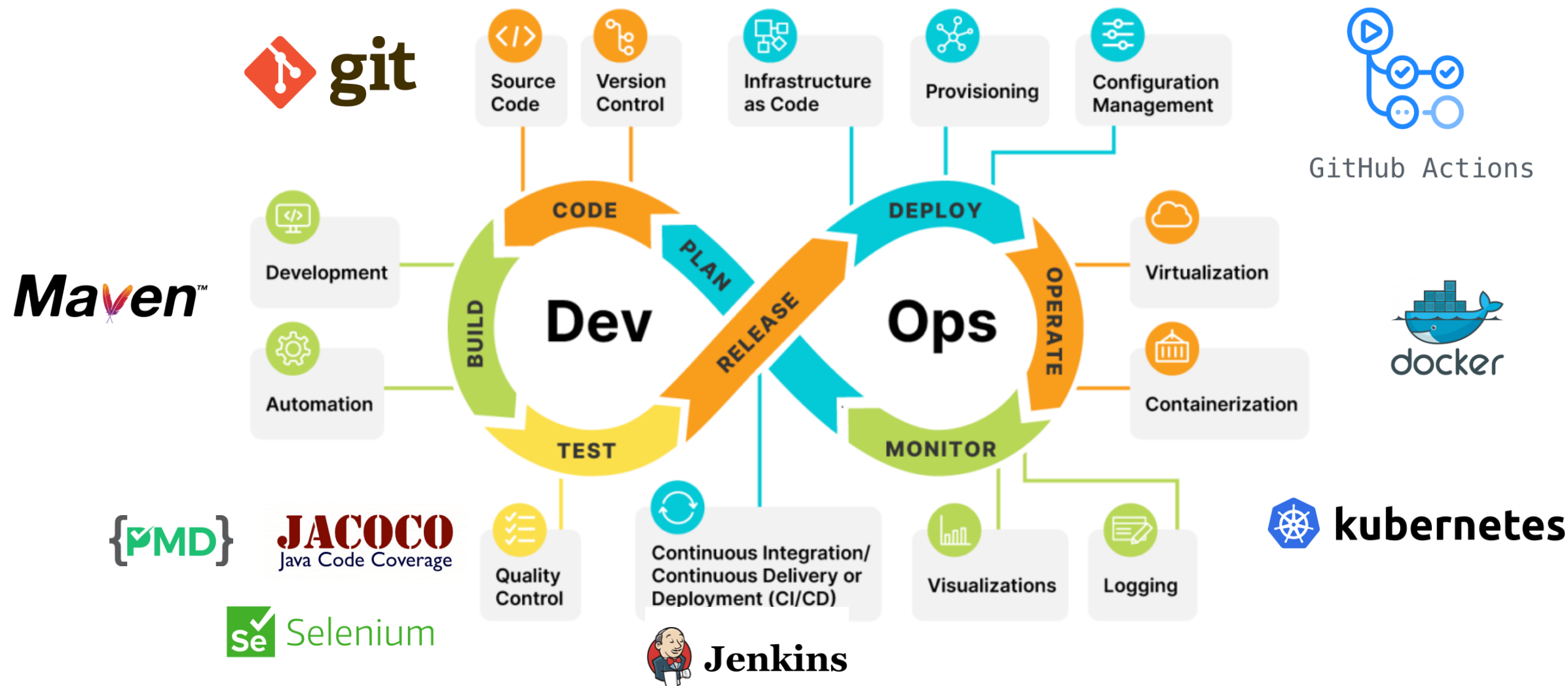
- Collaboration: Bridges the gap between developers and IT operations.
- Automation: Automates builds, testing, and deployments.
- Monitoring & Feedback: Uses real-time monitoring to improve system performance and user experience.

Benefits

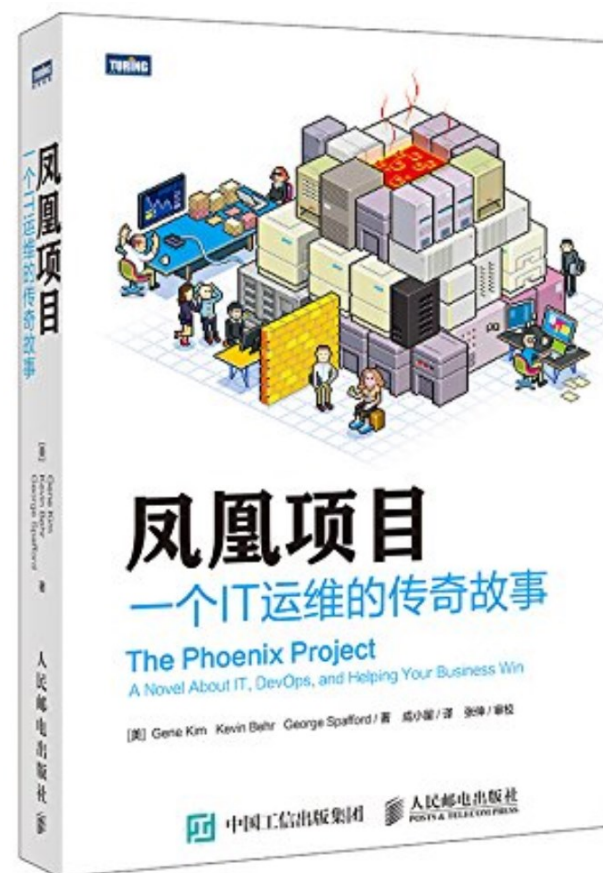
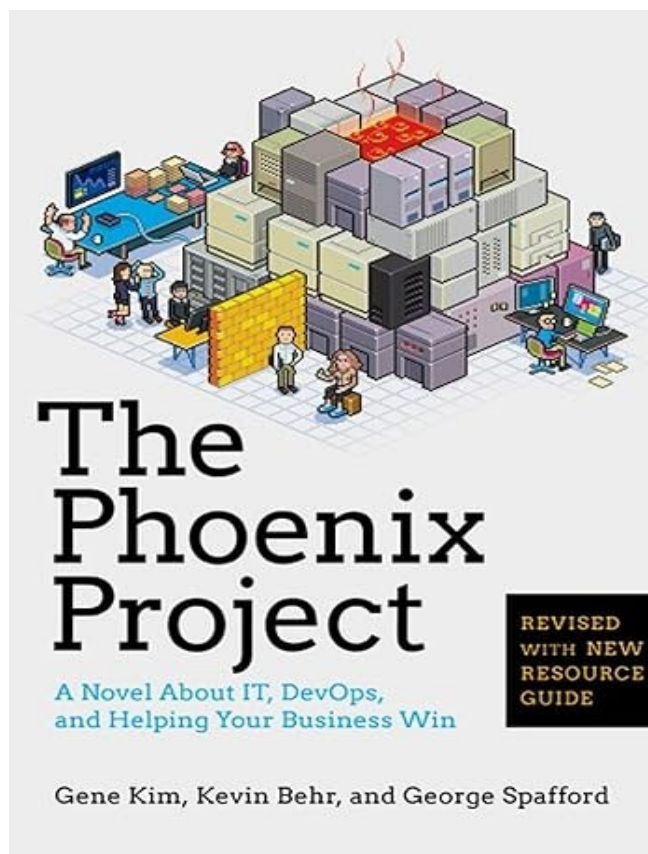
- Faster software releases
- Faster feedback
- Improved system reliability and stability
- Reduced deployment failures and rollback times
- Increased efficiency and resource utilization



OUR ROADMAP



READINGS



A business novel that reads like a thriller, following an IT manager's desperate race to save his company from collapse, revealing how the principles of DevOps can transform not just your technology, but your entire organization.



NEXT

- Software Requirements